OmniLink Pac-Man AI Agent System: Complete Technical Documentation

Executive Summary

This project implements an intelligent autonomous Pac-Man game system that combines a Python-based game engine with sophisticated TypeScript AI agents running in a web browser environment. The system uses the OmniLink platform to enable browser-based AI agents to control a classic Pac-Man game through HTTP REST APIs and MQTT messaging protocols. The project features multiple AI agent implementations with increasing sophistication, from basic BFS pathfinding to advanced multi-step lookahead trajectory prediction algorithms.

Key Technologies: Python (Pygame), TypeScript (Browser), HTTP REST API, MQTT Protocol, BFS Algorithms, Game AI

Target Users: AI researchers, game developers, robotics students, autonomous systems engineers

---

Table of Contents

- Project Overview and Architecture
- System Components
- AI Agent Implementations
- Communication Protocols
- Installation and Setup
- Running the System
- Troubleshooting
- Technical Details

---

1. Project Overview and Architecture

1.1 What is This Project?

This project demonstrates **autonomous decision-making in a real-time game environment**. It consists of three main layers:

1. **Python Game Backend** - A fully functional Pac-Man game with classic mechanics (ghosts, pellets, power-pellets, lives, levels, scoring)
2. **HTTP/MQTT Communication Layer** - A server wrapper that exposes game state and accepts control commands
3. **Browser-Based AI Agents** - TypeScript agents running in OmniLink platform that make intelligent movement decisions

1.2 System Architecture

Figure 1: System Architecture Diagram

Data Flow:

1. Python game runs on localhost, renders Pygame window
2. `server_wrapper.py` wraps the game and provides HTTP API on port 5000
3. TypeScript agent runs in browser (OmniLink platform)
4. Agent polls `GET http://localhost:5000/data` every 10-60ms
5. Server responds with complete game state (player position, ghost positions, pellets, maze structure)
6. Agent computes optimal move using AI algorithms
7. Agent posts chosen direction to `POST http://localhost:5000/callback`
8. Server updates game with agent's decision
9. Game continues at 60 FPS with agent-controlled Pac-Man

MQTT Integration:

- MQTT broker runs on localhost:1883 (standard) or localhost:9001 (WebSocket)
- Server publishes game status to `olink/context` topic every 20 seconds

- Agents subscribe to `olink/commands` topic for pause/resume control
- Agents can publish commands to control game state

1.3 Why This Architecture?

Separation of Concerns: Game logic (Python) is completely separate from AI logic (TypeScript), allowing independent development and testing.

Platform Compatibility: OmniLink runs in browsers, making the AI agents portable across devices without Python dependencies.

Real-time Performance: Polling at 10-60ms provides near-instantaneous response to game state changes while maintaining 60 FPS game loop.

Extensibility: Multiple AI agent implementations can compete against the same game backend.

---

2. System Components

2.1 Python Game Engine (`pacman-4.py`)

Purpose: Core Pac-Man game implementation with classic arcade mechanics.

Key Features:

- Full maze rendering with walls, pellets, power-pellets
- Four ghosts with distinct AI behaviors (Blinky, Pinky, Inky, Clyde)
- Ghost modes: SCATTER, CHASE, FRIGHTENED, EATEN
- Tunnel wrapping (horizontal screen wrap on specific rows)
- Multi-level progression with increasing difficulty
- Lives system and score tracking
- High score persistence to file
- Pause/resume functionality
- Resizable window with smooth scaling

Technical Highlights:

- Built with Pygame library
- 60 FPS game loop with delta-time physics
- BFS pathfinding for ghost navigation
- Tile-based movement system (20x20 pixel tiles)
- Character-based maze representation (`.` = pellet, `o` = power-pellet, `#` = wall)

Ghost Behaviors:

Ghost	Chase Target
Blinky (Red)	Direct chase - targets Pac-Man's exact position
Pinky (Pink)	Ambush - targets 4 tiles ahead of Pac-Man's direction
Inky (Cyan)	Complex - uses vector from Blinky to 2-ahead point, doubled
Clyde (Orange)	Proximity-based - chases when far, retreats to corner when close

Table 1: Classic ghost targeting strategies

2.2 Server Wrapper (`server_wrapper-6.py`)

Purpose: HTTP REST API server that bridges the Python game with browser-based agents.

Architecture:

- Runs three concurrent threads: HTTP server, MQTT client, and Pygame game loop
- HTTP server on port 5000 (daemon thread)
- MQTT client connects to localhost:1883
- Game runs in main thread with rendering

API Endpoints:

GET `/data` - Game State Polling

Returns JSON with complete game state:

- `command`: "ACTIVATE" (game running) or "IDLE" (paused/menu)

- version: Incrementing version number for synchronization
- payload: JSON string containing complete game state

Game State Structure:

- player: {x, y, dir} - Pac-Man position and direction
- ghosts: Array of {name, x, y, state}
- pellets: Array of [x, y] coordinates
- powerpellets: Array of [x, y] coordinates
- walkable: 2D grid (list of strings, "1"=walkable, "0"=wall)
- tunnelrows: Array of row indices that wrap horizontally
- score, lives, level, mode, gamestate
- gridwidth, gridheight, pelletsleft

POST /callback - Movement Command

Accepts JSON with agent's chosen direction:

```
{
"action": "UP" | "DOWN" | "LEFT" | "RIGHT" | "STOP",
"version": 123,
"timestamp": "2026-03-10T15:57:00.000Z"
}
```

Server updates player.nextdir to execute agent's decision.

MQTT Topics:

- olink/commands (subscribe) - Receives pause/resume commands
- olink/context (publish) - Broadcasts game summary every 20 seconds

Command Parsing:

The server intelligently parses various command formats:

- Plain text: "pause", "resume"
- JSON: {"command": "pause"}, {"action": "resume"}
- Quoted JSON: {"command":"pause"}
- Case-insensitive with flexible key names

2.3 AI Player (Local) (`ai_player-3.py`)

Purpose: Simple local AI for standalone testing without external agents.

Strategy: Basic BFS-based pellet collection with danger avoidance.

- Marks 5x5 radius around active ghosts as "danger zones"
- Chases frightened ghosts when available
- Uses BFS to find nearest safe pellet
- Avoids reversing direction unless necessary

Note: Only active when `AIENABLED = True` in `pacman-4.py`. Should be `False` when using external TS agents.

2.4 TypeScript AI Agents (Browser-Based)

Three progressively sophisticated agent implementations:

2.4.1 Advanced Agent (`advanced_agent.ts`)

Algorithm: BFS pathfinding with three behavioral modes.

Decision Logic:

1. **FLEE Mode** (ghost within 10 tiles)
    - Evaluate all valid directions
    - For each direction, perform BFS from that tile
    - Calculate minimum distance to all active ghosts
    - Choose direction that maximizes distance from nearest ghost
    - Add small pellet bonus (0.5 points) for tiebreaking
2. **CHASE Mode** (frightened ghost exists)
    - BFS to nearest frightened ghost
    - Return first-step direction toward target
    - Avoid reversing direction when possible
3. **HUNT Mode** (default)
    - Prefer power pellets when ghost within 12 tiles
    - BFS to closest reachable pellet
    - Return first-step direction

Configuration:

- `POLLDELAYMS = 15` - Very fast polling (66 updates/second)
- `FLEERADIUS = 10` - Conservative distance threshold
- `COMMITFRAMES = 5` - Brief directional commitment for smooth movement

2.4.2 Agent-2 (`agent-2.ts`)

Algorithm: Refined BFS with tighter reactivity.

Improvements over Advanced Agent:

- Faster polling (60ms intervals)
- Shorter flee radius (6 tiles) - more aggressive pellet collection
- Enhanced danger scoring with exponential penalties
- Better handling of tunnel wrapping edge cases

Key Difference: Balances safety vs. score more aggressively, allowing closer approaches to ghosts for pellet collection.

2.4.3 Perfect Agent (`perfect_agent-5.ts`)

Algorithm: Multi-step lookahead with ghost trajectory prediction.

Core Innovations:

1. **Ghost Trajectory Prediction** (4 steps ahead)
    - Simulate each ghost advancing toward Pac-Man using BFS
    - Build "future threat map" showing when/where ghosts will be
    - Apply time-weighted danger penalties to predicted positions
2. **Corridor Escape Analysis**
    - Before entering any corridor, check exit connectivity
    - If corridor has only 1 exit (dead-end), apply heavy penalty
    - Estimate: "Can I traverse and exit before ghost arrives?"
3. **Multi-Step Minimax Rollout** (3 steps deep)
    - For top-2 candidate directions (close scores)
    - Simulate 3 steps: agent picks best move each step, ghosts advance
    - Evaluate terminal board state danger
    - Choose candidate with safer 3-step outcome
4. **Aggressive Power-Pellet Seeking**
    - If any ghost within 12 tiles, power pellets get +180 weight

    - Place power pellets first in target list
    - Strong incentive to get powered up when threatened
5. **Panic Wall** (ghost ≤ 3 tiles)
    - Override all scoring, pure flee mode
    - Pick direction that maximizes minimum ghost distance
    - No pellet consideration, survival only

Scoring Weights:

Factor	Weight	Description
WPELLET	70	Base pellet attractiveness
WPOWER	180	Power pellet bonus when threatened
WCHASE	220	Chasing frightened ghosts
WDANGER	200	Danger penalty (exponential)
WFUTURE	40	Penalty for predicted ghost positions
WTRAP	60	Dead-end corridor penalty

Table 2: Perfect Agent scoring parameters

Configuration:

- `POLLDELAYMS = 10` - Ultra-fast polling (100 updates/second)
- `PANICDIST = 3` - Emergency flee threshold
- `FLEERADIUS = 10` - Danger consideration zone
- `LOOKAHEADSTEPS = 4` - Ghost prediction depth

Why "Perfect"?

This agent achieves near-optimal play by:

- Predicting ghost movements (not just reacting)
- Evaluating trap scenarios before committing
- Using minimax-style lookahead for close decisions
- Dynamically adjusting strategy based on threat level

---

3. Communication Protocols

3.1 HTTP REST API

Base URL: `http://localhost:5000`

Agent Polling Loop (TypeScript):

```
async function agentLoop() {
const res = await fetch(${APIURL}/data);
const wrapper = await res.json();

if (wrapper.command === "ACTIVATE") {
const state = JSON.parse(wrapper.payload);
const move = chooseMove(state);

  await fetch(`${APIURL}/callback`, {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({
      action: move,
      version: wrapper.version,
      timestamp: new Date().toISOString()
    })
  });

}

setTimeout(agentLoop, POLLDELAYMS);
}
```

Request/Response Cycle:

1. Agent polls `/data` endpoint
2. Server increments version counter
3. Server serializes current game state to JSON
4. Agent receives state, computes move (10-50ms)
5. Agent posts move to `/callback`
6. Server applies move to `player.nextdir`
7. Game loop executes move at next frame (16ms @ 60fps)

Synchronization: Version numbers ensure agent and server stay synchronized, detecting disconnections or missed updates.

3.2 MQTT Protocol

Broker Configuration:

- Standard MQTT: `localhost:1883`
- WebSocket MQTT: `ws://localhost:9001`

Topic Structure:

`olink/commands` (Server subscribes, Agent publishes)

Agent sends pause/resume commands:

```
{"command": "pause"}
{"command": "resume"}
```

`olink/context` (Server publishes, Agent subscribes)

Server broadcasts game summary every 20 seconds:

```
{
"topic": "game/summary",
"score": 1240,
"lives": 5,
"level": 2,
"state": "PLAY",
"mode": "CHASE",
"pelletsleft": 178,
"timestamp": "2026-03-10T15:57:00Z"
}
```

Agent MQTT Integration:

```
const client = mqtt.connect('ws://localhost:9001', {
clientId: 'pacman-perfect-' + Date.now()
});

client.on('connect', () => {
client.subscribe('olink/commands');
});
```

```
client.on('message', (topic, message) => {
const cmd = JSON.parse(message.toString());
if (cmd.command === 'pause') isPaused = true;
if (cmd.command === 'resume') isPaused = false;
});

// Publish commands
client.publish('olink/commands', JSON.stringify({command: 'pause'}));
```

Use Case: MQTT enables external control and monitoring of the game, useful for:

- Dashboard applications
- Multi-agent coordination
- Remote game state monitoring
- Debugging and logging

---

4. Installation and Setup

4.1 Prerequisites

Python Requirements:

- Python 3.8 or higher
- pip package manager

System Requirements:

- Operating System: Windows, macOS, or Linux
- RAM: 2GB minimum
- Display: 800x600 minimum resolution

MQTT Broker:

- Mosquitto MQTT broker (recommended)
- Or any MQTT broker supporting WebSockets

OmniLink Platform Access:

- Browser with JavaScript enabled
- OmniLink account and access

- Tool creation permissions

4.2 Python Dependencies Installation

Step 1: Create virtual environment (recommended)

```
python -m venv pacman-env
source pacman-env/bin/activate # Linux/Mac
pacman-env\Scripts\activate # Windows
```

Step 2: Install required packages

```
pip install pygame paho-mqtt
```

Package Purposes:

- `pygame` (v2.0+) - Game rendering and event handling
- `paho-mqtt` (v1.6+) - MQTT client library

Verify Installation:

```
python -c "import pygame; import paho.mqtt.client as mqtt;
print('Success')"
```

4.3 MQTT Broker Setup

Option A: Mosquitto (Recommended)

Linux:

```
sudo apt-get install mosquitto mosquitto-clients
sudo systemctl start mosquitto
sudo systemctl enable mosquitto
```

macOS:

```
brew install mosquitto
brew services start mosquitto
```

Windows:

1. Download from https://mosquitto.org/download/
2. Install to C:\ Program Files\ mosquitto
3. Run `mosquitto.exe` or install as service

Configuration (`/etc/mosquitto/mosquitto.conf`):

```
listener 1883
listener 9001
protocol websockets
allow_anonymous true
```

Option B: Docker

```
docker run -d -p 1883:1883 -p 9001:9001
--name mosquitto
eclipse-mosquitto
```

Test MQTT Connection:

```
Terminal 1 (Subscribe):
mosquitto_sub -h localhost -t test/topic

Terminal 2 (Publish):
mosquitto_pub -h localhost -t test/topic -m "Hello MQTT"
```

You should see "Hello MQTT" appear in Terminal 1.

4.4 File Organization

Recommended Directory Structure:

```
pacman-ai-project/
├── pacman-4.py # Core game engine
├── server_wrapper-6.py # HTTP/MQTT server
├── ai_player-3.py # Local AI (optional)
├── advanced_agent.ts # Basic BFS agent
├── agent-2.ts # Refined BFS agent
├── perfect_agent-5.ts # Advanced lookahead agent
├── pacman_hiscore.txt # Auto-generated high score
└── README.md # This documentation
```

File Permissions:

Ensure all Python files are executable:

```
chmod +x *.py # Linux/Mac
```

---

5. Running the System

5.1 Starting the Game Backend

Step 1: Ensure MQTT broker is running

Check if mosquitto is running

ps aux | grep mosquitto # Linux/Mac
netstat -an | findstr 1883 # Windows

Step 2: Start the server wrapper

python server_wrapper-6.py

Expected Output:

[HTTP] API server running on port 5000
[MQTT] Client started (broker=localhost:1883)
[MQTT] Connected to broker at localhost:1883
[MQTT] Subscribed to olink/commands
[Game] Initialising Pac-Man game...
[Game] Ready. Waiting for agent commands

Step 3: Verify game window appears

- Pygame window opens showing Pac-Man maze
- Title screen: "PAC-MAN PYTHON - Press Enter/Space to start"
- **DO NOT press Enter yet** - wait for agent to connect

Step 4: Test API manually (optional)

Open browser to `http://localhost:5000/data`

You should see JSON response:

{
"command": "IDLE",
"version": 1,
"payload": "{"type":"state", ...}"
}

5.2 Deploying Agent to OmniLink

Step 1: Log into OmniLink platform

Navigate to your OmniLink workspace in browser.

Step 2: Create new Tool

- Click "Create Tool" or "New Tool"
- Tool Type: JavaScript/TypeScript Tool
- Execution: Isolated Worker (ESM)

Step 3: Copy agent code

Choose one agent implementation:

- `advanced_agent.ts` - For learning/testing
- `agent-2.ts` - For improved performance
- `perfect_agent-5.ts` - For best gameplay

Copy entire file contents into OmniLink code editor.

Step 4: Configure tool settings

- Name: "Pac-Man AI Agent"
- Description: "Autonomous Pac-Man player using BFS/Lookahead"
- Permissions: Allow network access to localhost

Step 5: Save and activate tool

Click "Save" then "Activate" or "Run"

Expected Console Output:

OmniLink Perfect Agent v4 (Lookahead Trajectory)

[CONFIG] API: http://localhost:5000 (10ms poll)
[CONFIG] MQTT: ws://localhost:9001
[CONFIG] PANICDIST: 3 tiles
[CONFIG] FLEERADIUS: 10 tiles
[CONFIG] LOOKAHEAD: 4 ghost steps
[CONFIG] Weights: PELLET=70 POWER=180 CHASE=220

```
DANGER=200
[INFO] globalThis.pauseGame/resumeGame available

[MQTT] Connected to ws://localhost:9001
[MQTT] Subscribed to olink/commands
[AGENT] IDLE (v2)
[AGENT] IDLE (v3)
...
```

Agent is now polling, waiting for game to start.

5.3 Starting Gameplay

In Pygame window: Press **Enter** or **Space** to start game

Agent immediately takes control:

```
[AGENT] Game state received (v4)
[AI] HUNT pellet (dist=5) -> RIGHT (pellets_left=244)
[AGENT] Posted move: RIGHT
[GAME] Score: 0 -> 10 (+10)
[AI] HUNT pellet (dist=4) -> RIGHT (pellets_left=243)
[AI] FLEE (ghost: BLINKY dist=8) -> UP
...
```

Game Progression:

- Agent autonomously navigates maze
- Collects pellets and power-pellets
- Flees from active ghosts
- Chases frightened ghosts after eating power-pellets
- Advances through levels as maze clears
- Loses lives on ghost collisions
- Game over at 0 lives

5.4 Control Commands

During Gameplay:

- **P** - Pause/Resume (Python window)
- **ESC** - Quit game
- **Arrow Keys** - Manual override (if agent disconnected)

MQTT Commands (from browser console or external tool):

```
// In OmniLink tool or browser with MQTT client
pauseGame(); // globalThis.pauseGame()
resumeGame(); // globalThis.resumeGame()
```

From Command Line:

Pause game

```
mosquitto_pub -h localhost -t olink/commands -m
'{"command":"pause"}'
```

Resume game

```
mosquitto_pub -h localhost -t olink/commands -m
'{"command":"resume"}'
```

5.5 Monitoring Game State

Python Console Output:

```
[GAME] Score: 1240 -> 1250 (+10)
[GAME] Lives: 7 -> 6
[GAME] Level: 1 -> 2
[MQTT] Published to olink/context (score=1250 lives=6 level=2
state=PLAY)
```

Agent Console Output:

```
[AI] HUNT pellet dist=3 -> UP (pellets_left=178)
[AI] FLEE (ghost: BLINKY dist=4) -> LEFT
[AI] CHASE frightened ghost -> RIGHT (dist=5)
[AI] PANIC FLEE (ghost dist=2)
[AI] Rollout tiebreak: LEFT beats RIGHT (120 vs 180)
```

MQTT Monitoring:

Subscribe to game state broadcasts

mosquitto_sub -h localhost -t olink/context -v

Output:

olink/context {"topic":"game/summary","score":2340,"lives":5,...}

---

6. Troubleshooting

6.1 Common Issues

Problem: "Connection refused" on port 5000

Cause: Server wrapper not running or port blocked

Solution:

- Verify `server_wrapper-6.py` is running
- Check firewall allows localhost:5000
- Try different port in both server and agent code

Problem: Agent spams "AGENT: IDLE"

Cause: Game not started (still on title screen)

Solution: Press Enter/Space in Pygame window to start game

Problem: "MQTT Client not ready" warnings

Cause: MQTT broker not running or wrong configuration

Solution:

- Start Mosquitto broker
- Check broker accepts WebSocket connections on port 9001
- Verify `mosquitto.conf` has WebSocket listener enabled
- Test with `mosquitto_sub` and `mosquitto_pub`

Problem: Pac-Man doesn't move or moves randomly

Cause: Agent not connected or crashed

Solution:

- Check browser console for agent errors
- Verify agent code is running in OmniLink
- Check network tab for API request failures
- Restart agent tool

Problem: High CPU usage

Cause: Polling too fast or BFS not optimized

Solution:

- Increase `POLLDELAYMS` in agent (try 50ms)
- Close other applications
- Use `advanced_agent.ts` instead of `perfect_agent-5.ts` (simpler algorithm)

Problem: Agent gets stuck in corners

Cause: Reverse direction avoidance too aggressive

Solution:

- Check `forbidReverse` logic allows reversing when necessary
- Reduce `COMMITFRAMES` value
- Ensure BFS fallback handles dead-ends

Problem: ImportError for pygame or paho.mqtt

Cause: Packages not installed or wrong Python environment

Solution:

Activate virtual environment if using one

source pacman-env/bin/activate

Reinstall packages

pip install --upgrade pygame paho-mqtt

Verify installation

pip list | grep -E "pygame|paho-mqtt"

6.2 Debugging Tips

Enable Detailed Logging:

In agent TypeScript files, set logging flags to `true`:

```
const LOG_MOVE = true;
const LOG_DANGER = true;
const LOG_IDLE = true;
const LOG_MQTT = true;
const LOG_ERRORS = true;
```

Inspect Game State:

Add logging in server wrapper `server_wrapper-6.py`:

```
def build_game_state(game):
state = {...}
print(f"[DEBUG] Player at ({state['player']['x']}, {state['player']['y']})")
print(f"[DEBUG] Ghosts: {len(state['ghosts'])}, Pellets:
{len(state['pellets'])}")
return state
```

Monitor API Calls:

Use browser DevTools Network tab to inspect:

- Request frequency (should match `POLLDELAYMS`)
- Response times (should be <50ms)
- Status codes (200 OK expected)
- Payload sizes (typical: 2-5KB per response)

Test Standalone Game:

Run without server wrapper to isolate game issues:

In pacman-4.py

AIENABLED = True # Use built-in AI

Run directly

python pacman-4.py

MQTT Debugging:

Monitor all MQTT traffic:

mosquitto_sub -h localhost -t '#' -v

6.3 Performance Optimization

For Low-End Systems:

- Use `advanced_agent.ts` (simplest algorithm)
- Increase `POLLDELAYMS` to 50-100ms
- Reduce Pygame window size
- Disable MQTT publisher loop in `server_wrapper-6.py`

For Maximum Performance:

- Use `perfect_agent-5.ts` with `POLLDELAYMS = 10`
- Enable hardware acceleration in browser
- Close unnecessary browser tabs
- Run on dedicated Python environment

---

7. Technical Details

7.1 Coordinate System

Grid-Based System:

- Origin (0,0) at top-left corner
- X-axis increases rightward
- Y-axis increases downward
- Each tile is 20x20 pixels
- Typical maze: 28 tiles wide × 31 tiles tall

Position Representation:

- Integer coordinates: tile indices (e.g., (13, 22))
- Float coordinates: pixel-perfect positions (e.g., (13.5, 22.8))
- `player.x`, `player.y` are floats
- BFS algorithms use `int(x)`, `int(y)` for tile lookups

Tunnel Wrapping:

Specific rows allow horizontal wrapping:

- If `y in tunnelrows` and `x < 0`, wrap to `x = width - 1`
- If `y in tunnelrows` and `x >= width`, wrap to `x = 0`
- Ghosts and player can teleport across screen edges

7.2 BFS Algorithm Details

Breadth-First Search guarantees shortest path in unweighted grids.

Implementation:

def bfs_distances(sx, sy, maze, stop_at=None):
dist = {}
queue = [(sx, sy)]
dist[(sx, sy)] = 0

```
while queue:
    x, y = queue.pop(0)
    d = dist[(x, y)]
```

```
    for (nx, ny, _) in maze.neighbors(x, y):
        if (nx, ny) not in dist:
            dist[(nx, ny)] = d + 1
            queue.append((nx, ny))

return dist
```

Time Complexity: O(V + E) where V = tiles, E = edges (typically 4V)

Space Complexity: O(V) for distance map

Optimizations in Agents:

- Early termination when all targets found
- Distance caching for multiple queries
- Limit BFS depth (e.g., `maxDist=40`) for performance

7.3 Agent Decision-Making Pipeline

Step 1: State Acquisition (1-5ms)

- Poll API endpoint
- Parse JSON payload
- Validate state version

Step 2: Threat Assessment (2-10ms)

- Identify active vs frightened ghosts
- Run BFS from each ghost position
- Compute distances to player
- Classify threat level (PANIC, FLEE, normal)

Step 3: Target Selection (1-5ms)

- If PANIC: no target, pure avoidance
- If FLEE: target = direction maximizing ghost distance
- If CHASE: target = nearest frightened ghost
- If HUNT: target = nearest pellet (prefer power pellets if threatened)

Step 4: Pathfinding (5-20ms)

- BFS from player to target
- Backtrack to find first-step direction
- Apply constraints (avoid reverse, respect walls)

Step 5: Advanced Analysis (Perfect Agent only, 10-30ms)

- Ghost trajectory prediction (4 steps)
- Future threat map construction
- Corridor escape analysis
- Multi-step rollout for top-2 candidates

Step 6: Move Execution (<1ms)

- Format JSON action payload
- POST to /callback endpoint
- Update internal state

Total Latency: 20-70ms (depends on agent complexity)

7.4 Ghost State Machine

States:

State	Behavior
SCATTER	Move toward designated corner (home corner)
CHASE	Active pursuit of Pac-Man using targeting strategy
FRIGHTENED	Random movement, slow speed (3.0), vulnerable to Pac-Man
EATEN	Return to spawn point at high speed (9.0), ignore Pac-Man

Table 3: Ghost states and behaviors

Mode Schedule (classic Pac-Man timing):

1. SCATTER (20 seconds) - Initial dispersion
2. CHASE (8 seconds)

3. SCATTER (20 seconds)
4. CHASE (8 seconds)
5. SCATTER (20 seconds)
6. CHASE (infinite)

Frightened State Trigger:

- Pac-Man eats power-pellet
- Duration: 20 - (level - 1) * 2.5 seconds (minimum 3 seconds)
- Ghosts reverse direction immediately
- Speed reduced to 3.0 (vs player 7.5)
- If eaten by Pac-Man, enter EATEN state

7.5 Scoring System

Point Values:

Action	Points
Eat pellet (.)	10
Eat power-pellet (o)	50
Eat 1st frightened ghost	200
Eat 2nd frightened ghost (chain)	400
Eat 3rd frightened ghost (chain)	800
Eat 4th frightened ghost (chain)	1600

Table 4: Scoring values

Ghost Chain Multiplier:

- Resets when power-pellet effect ends
- Incentivizes eating multiple ghosts per power-pellet
- Maximum chain: 4 ghosts = 3000 points total

High Score Persistence:

- Stored in `pacman_hiscore.txt`
- Updated whenever current score exceeds saved high score
- Survives game restarts

7.6 Level Progression

Level Completion:

- Triggered when `maze.pellets_total == 0`
- Level increments: `self.level += 1`
- Maze resets with full pellet set
- Player and ghosts respawn at starting positions

Difficulty Scaling:

Player Speed: `7.5 + 0.1 * (level - 1)` tiles per second

Ghost Speed: `4.0 + 0.05 * (level - 1)` tiles per second

Frightened Duration: `max(3.0, 20.0 - (level - 1) * 2.5)` seconds

Analysis:

- Player speed increases twice as fast as ghosts
- Player always remains faster than ghosts (margin decreases)
- By level 10: Player 8.4 tps vs Ghost 4.45 tps (still ~2x faster)
- Frightened duration drops to minimum (3s) by level 8

7.7 OmniLink Platform Integration

Execution Environment:

- Isolated JavaScript Worker (ESM modules)
- No DOM access (pure computation)
- Network access via `fetch()` API
- WebSocket support for MQTT

Global APIs Available:

- `globalThis.fetch` - HTTP requests
- `globalThis.mqtt` - MQTT client library (injected)
- `globalThis.console` - Logging to OmniLink console
- `setTimeout`, `setInterval` - Timing functions

Agent Lifecycle:

1. Agent code executes on tool activation

2. Bootstrap code runs immediately (console.log configuration)
3. MQTT initialization (initMqtt())
4. HTTP polling loop starts (runLoop())
5. Agent runs until tool deactivation or error

Error Handling:

```
try {
const res = await fetch(${API_URL}/data);
if (!res.ok) throw new Error(HTTP ${res.status});
// Process...
} catch (err) {
if (LOG_ERRORS) {
const msg = err instanceof Error ? ${err.name}: ${err.message} :
String(err);
console.error('[AGENT] Error:', msg);
}
}
```

Resource Limits:

- CPU: Shared with other tools, no hard limit
- Memory: Typically 100-500MB per tool
- Network: No rate limits on localhost
- Execution time: 30-second timeout per async operation

---

8. Advanced Topics

8.1 Extending the AI Agent

Adding New Behavior Modes:

```
function chooseMove(state: GameState): Dir {
// Existing modes: FLEE, CHASE, HUNT

// Add AMBUSH mode: predict player path, cut them off
if (shouldAmbush(state)) {
return computeAmbushMove(state);
}
```

```
// Add DEFENSIVE mode: guard remaining power pellets
if (powerPelletsInDanger(state)) {
return guardPowerPellet(state);
}

// Fall back to existing logic
return existingChooseMove(state);
}
```

Implementing Machine Learning:

- Collect training data: `(state, action, reward)` tuples
- Export to Python for training (DQN, PPO, etc.)
- Load trained model weights into agent
- Replace BFS logic with neural network inference

8.2 Multi-Agent Systems

Cooperative Agents:

Run multiple agent instances controlling different aspects:

- Agent 1: Pellet collection specialist
- Agent 2: Ghost evasion specialist
- Coordinator: Combines recommendations via voting or weighting

Competitive Agents:

- Modify game for 2-player mode
- Each agent controls one Pac-Man
- Compare scores and survival times

8.3 Performance Benchmarking

Metrics to Track:

- Average score per game
- Survival time (seconds before first death)
- Pellet collection efficiency (pellets/second)
- Ghost avoidance success rate
- Power-pellet utilization (ghosts eaten per power-pellet)

Automated Testing Framework:

test_agents.py

```
def run_trial(agent_type, num_games=100):
results = []
for i in range(num_games):
score = run_game_with_agent(agent_type)
results.append(score)
return statistics.mean(results), statistics.stdev(results)

print(f"Advanced Agent: {run_trial('advanced')}")
print(f"Perfect Agent: {run_trial('perfect')}")
```

8.4 Visualization and Analysis

Add Heatmaps:

Visualize where agent spends time:

In server_wrapper.py, track positions

```
position_counts = defaultdict(int)

def build_game_state(game):
pos = (int(game.player.x), int(game.player.y))
position_counts[pos] += 1
# ... rest of function
```

Export heatmap data, visualize with matplotlib or in browser.

Decision Replay:

Log all decisions with timestamps:

```
const decisions = [];
decisions.push({
timestamp: Date.now(),
```

```
state: state,
move: chosenMove,
score: moveScore
});
// Export for analysis
```

Replay decisions to debug failures or optimize scoring weights.

Conclusion

This project demonstrates a complete autonomous game-playing system combining classical AI (BFS pathfinding), modern web technologies (HTTP/MQTT APIs), and sophisticated decision-making algorithms (lookahead prediction). The modular architecture allows experimentation with different AI approaches while maintaining a clean separation between game logic and agent intelligence.

Key Takeaways:

- **Real-time AI**: Sub-100ms decision cycles enable responsive gameplay
- **Pathfinding Fundamentals**: BFS provides optimal solutions for grid navigation
- **Threat Modeling**: Predictive algorithms outperform reactive strategies
- **API Design**: RESTful polling + MQTT events = flexible communication
- **Incremental Complexity**: Three agent tiers demonstrate progressive sophistication

Next Steps for Learning:

1. Start with `advanced_agent.ts` to understand basic BFS
2. Analyze `perfect_agent-5.ts` to see advanced techniques
3. Modify scoring weights and observe behavioral changes
4. Implement your own decision strategies
5. Extend to other games (Snake, Maze, Grid World problems)

Useful Resources:

- Pac-Man Ghost AI: https://gameinternals.com/understanding-pac-man-ghost-behavior
- BFS/Pathfinding: Introduction to Algorithms (Cormen et al.)
- Game AI: "Programming Game AI by Example" (Mat Buckland)
- MQTT Protocol: https://mqtt.org/

Appendix A: Quick Reference Commands

Start System:

Terminal 1: Start MQTT broker

mosquitto -c /etc/mosquitto/mosquitto.conf

Terminal 2: Start game backend

python server_wrapper-6.py

Browser: Deploy agent to OmniLink

Copy perfect_agent-5.ts to OmniLink tool editor

Activate tool

Pygame window: Press Enter to start

Control Commands:

Pause via MQTT

mosquitto_pub -h localhost -t olink/commands -m '{"command":"pause"}'

Resume via MQTT

mosquitto_pub -h localhost -t olink/commands -m '{"command":"resume"}'

Monitor game state

mosquitto_sub -h localhost -t olink/context

Testing:

Test API endpoint

curl http://localhost:5000/data

Test callback endpoint

curl -X POST http://localhost:5000/callback
-H "Content-Type: application/json"
-d '{"action":"UP","version":1,"timestamp":"2026-03-10T15:57:00.000Z"}'

Test MQTT pub/sub

mosquitto_sub -h localhost -t '#' -v &
mosquitto_pub -h localhost -t test -m hello

Appendix B: Configuration Parameters

Agent Configuration (TypeScript files):

Parameter	Advanced	Agent-2	Perfect
POLLDELAYMS	15	60	10
FLEERADIUS	10	6	10
PANICDIST	-	-	3
COMMITFRAMES	5	5	-
LOOKAHEADSTEPS	-	-	4

Table 5: Agent tuning parameters

Server Configuration (`server_wrapper-6.py`):

- `HTTP_PORT = 5000` - API server port
- `MQTT_BROKER = "localhost"` - MQTT broker address
- `MQTT_PORT = 1883` - Standard MQTT port (9001 for WebSocket)
- `CMD_TOPIC = "olink/commands"` - Command topic
- `CTX_TOPIC = "olink/context"` - Status broadcast topic
- `PUBLISH_EVERY = 20` - MQTT status interval (seconds)

Game Configuration (`pacman-4.py`):

- `TILE = 20` - Tile size in pixels
- `FPS = 60` - Target frame rate
- `AI_ENABLED = False` - Use built-in AI (set False for external agent)

Document Version: 1.0
Last Updated: March 10, 2026
Author: AI Systems Documentation Team

Target Audience: Computer Engineering Students, AI Researchers, Game Developers